Sara — A Numba-CUDA Ray Tracer

Matěj Zeman*
Supervisor: Ing. Radek Richtr, Ph.D.
Course: NI-PG1 — Počítačová Grafika 1

May 2026

Abstract

This report presents a simple ray tracer implemented in Python, utilizing Numba CUDA for GPU acceleration. The renderer supports most of the ray-tracing pipeline including diffuse textures, reflective and transmissive materials, area light integration, and ACES-based colour management with Intel Open Image Denoise [5] pass. A key contribution is the development and comparison of three BVH construction strategies: naive triangle enumeration, median splitting, and Surface Area Heuristic (SAH) with binning. The project also documents practical limitations of using a Python-to-CUDA JIT compiler.

1 Engine Structure & Implementation Challenges

1.1 Core Architecture

Sara is built around Numba's CUDA JIT compiler, which translates a subset of Python and NumPy into PTX. The rendering pipeline runs primarily on the GPU, with the CPU serving optional support tasks:

- **GPU** — Ray tracing (primary, secondary, and shadow rays), BVH traversal via per-thread stack, Möller–Trumbore intersection [2], material shading, HDR framebuffer, tonemapping, OIDN denoising.
- **CPU** — Scene loading via `tinyobjloader` (C++, MIT License) [7], BVH construction, optional post-processing (tonemapping, OIDN denoise, sRGB conversion).

Numba vs CUDA C++: This project is unusual because it implements a full path tracer through Python's JIT layer rather than native CUDA C++. Numba lacks the mature ecosystem of C++ ray tracers: no GPU-side sorting (forcing SAH BVH on CPU), no acceleration libraries, and no built-in occupancy analysis. Type inference silently promotes `float32` to `float64` through Python's scalar rules. Host-to-device transfers carry Python runtime overhead. Debugging is restricted to Nsight Compute [4] metrics and printed values, with no native GPU debugger. The kernel must be a single megakernel because Numba's launch model does not easily support multi-kernel pipelines with intermediate device buffers.

Scene data uses a *struct-of-arrays* layout in contiguous 1D NumPy arrays: BVH nodes (8 `float32` per node), triangles (9 `float32`: 3 vertices), and materials (14 columns: diffuse/specular/emissive/transmission RGB, roughness, IOR). The kernel uses a $16 \times 16 = 256$-thread block, with each thread rendering one pixel. Samples accumulate iteratively with HDR framebuffer averaged at the end. An ACEScg-to-sRGB colour pipeline handles gamut mapping with six tonemapper options (Section 5.1).

1.2 Implementation Challenges

- **No GPU sorting:** Numba lacks GPU sort primitives, forcing SAH BVH construction onto the CPU (e.g., 6.69 s for dragon).
- **Iterative traversal:** BVH traversal uses a per-thread local stack (`STACK_SIZE` = 64) to avoid recursion depth limits.
- **Register pressure & occupancy:** The megakernel carries many variables per thread. On Ampere (RTX 3090), high register pressure reduces occupancy versus Ada Lovelace (RTX 40-series) [3]. Block sizes above 16×16 (32×32 = 1024 threads) fail with `CUDA_ERROR_LAUNCH_OUT_OF_RESOURCES` because the kernel requires too many registers per thread to fit the full block on one SM. The 8×8 configuration yields marginally better throughput on the RTX 4070 Ti (11% faster) due to lower register pressure, confirming that 16×16 was a deliberate choice for higher occupancy at the cost of raw speed.
- **Float64 propagation:** The Python `float` type promotes NumPy expressions to `float64`, causing slow double-precision math on the GPU. This is mitigated using `float32()` casts and typed aliases in `constants.py`.
- **Precision and epsilons:** Three epsilon values prevent self-shadowing and division errors: `DET_EPSILON` (10^{-9}), `RAY_EPSILON` (10^{-5}), `BARYCENTRIC_EPSILON` (10^{-7}).
- **Numba classes:** Numba's `types.ClassType` support on the GPU is experimental and can cause performance regressions through hidden memory allocations and type dispatch overhead. The project avoids classes entirely, keeping the code close to C-style procedural code with explicit structs-of-arrays and simple function dispatch.

1.3 Divergence & Wavefront Architecture

Baseline profiling on the megakernel reveals significant warp divergence: Nsight Compute measurements show an

*Student, CTU FIT Prague, `zemanm40@fit.cvut.cz`

average of 8.38 active threads per warp (26.2% utilization) on the dragon scene, with 7.42 warp cycles per issued instruction. This divergence stems from workload variance—shadow rays are coherent (all threads test nearby geometry), reflective/transmissive rays diverge (threads follow different paths through the material graph), and BVH traversal is inherently incoherent due to chaotic ray directions.

An alternative *wavefront* architecture was implemented on the `gpu` branch to mitigate this. The two-pass approach enforces an operations budget in Pass 1: easy rays finish and terminate, while complex rays save their state to global memory. A compacted array of only unfinished rays is then processed in Pass 2, isolating divergent paths. This approach achieved 30% speedup on Ada Lovelace (RTX 4070 Ti) but failed on Ampere (RTX 3070/A100) due to register spilling, proving that the optimal architecture depends on the target silicon. However, wavefront compaction worsened spatial locality—Pass 2 active threads dropped to 4.68 per warp (14.6%) because the compacted array destroyed adjacent memory access patterns.

2 Advanced Methods

Beyond basic ray tracing: **Stochastic jittered sampling** samples each pixel `SAMPLES` times with uniform jitter within the pixel footprint. **Intel OIDN denoising** [5] uses the `RT` filter with a single `color` buffer in `hdr` mode, denoising HDR pixel colour directly. On a Python renderer limited to 10 SPP per second, OIDN makes 16 SPP rendering visually acceptable—denoising is not an optional enhancement but the primary reason this renderer produces practical results at all. The ACEScg colour pipeline operates in linear space for all shading, with multiple tonemapping strategies available (Section 5.1). A 3D LUT approach for the custom tonemapper was chosen to offload complex colour space conversions from the per-pixel GPU kernel, trading a small memory footprint for improved render time.

Numba's compilation model: Numba uses Python decorators (`@cuda.jit`, `fastmath=True`) evaluated at JIT time. The compiler infers types from NumPy dtypes, lowers to LLVM IR, then generates PTX. Dead code elimination prunes `if FLAG:` blocks at compile time. NumPy's promotion rules can elevate `float32` to `float64`. All literals are wrapped in `float32()`. Stochastic sampling uses per-thread seeds computed from pixel coordinates and sample index.

Memory hierarchy: The `cuda.shared.array` API allocates block-level shared memory, while `cuda.local.array` provides per-thread private storage. Exceeding register limits causes spills to global memory. `STACK_SIZE` is kept at 64 to prevent register exhaustion during BVH traversal.

3 Acceleration Data Structure

3.1 Methodology

Top-down BVH with three split selection strategies was implemented [1]. GPU-side BVH traversal is iterative, using a per-thread local stack. For each ray, the algorithm visits nodes in depth-first order, testing both children at internal nodes and all primitives at leaf nodes. Per-pixel metrics (node visits, triangle tests, shadow tests, hit status) are recorded for profiling.

3.2 Construction Strategies

Three strategies were implemented on the CPU (with `@njit` decorator):

1. **Naive** — All triangles in a single leaf node. Tests every triangle per ray. Impractical beyond a handful of primitives.
2. **Spatial Median** — Splits at the geometric midpoint along the longest axis. Produces a balanced tree quickly but ignores traversal cost asymmetry.
3. **SAH + Binning** — Minimises the cost function

$$C(s) = 1.0 + \frac{N_{\text{left}} \cdot A_{\text{left}} + N_{\text{right}} \cdot A_{\text{right}}}{A_{\text{parent}}}$$

   where A is child AABB surface area and N is the number of triangles in each child. The constant 1.0 represents the traversal cost of the current node. Binning ($B = 8$ bins per axis) avoids evaluating every split point, trading minor quality loss for significantly faster construction.

Exhaustive SAH evaluates every triangle as a split candidate ($O(N^2)$ per node). Binning maps triangles into $B = 8$ bins per axis with prefix/suffix accumulations, evaluating all $B - 1$ inter-bin splits in $O(B)$ time. This drops construction complexity from $O(N^2)$ to $O(N \cdot \text{depth})$. Table 4.1 shows exhaustive SAH improves only 13% over binned SAH (775 vs. 892 mean node visits). Dragon was not benchmarked with median-split or exhaustive SAH due to build time constraints.

The BVH builder is single-threaded:
Numba's `@njit(parallel=True)` CPU parallelism only supports flat `prange` loops, not dynamic task spawning, so the two independent subtrees cannot be distributed across the 16 available cores during construction. After construction, the tree is *flattened* into `bvh_nodes` (8 floats per node) and `tri_ids` arrays, enabling O(1) child access on the GPU.

4 Performance Results

Scene	Config	Build (s)	Node	Tri	Shadow	Traverse
bunny	sah-binning	4.16	892.42	77.26	19.82	29.78
bunny	median-split	5.31	2903.43	413.77	19.83	97.69
bunny	sah-exhaustive	508.92	775.11	79.97	19.82	25.70
box-spheres	sah-binning	3.39	454.18	52.85	13.34	15.26
box-spheres	median-split	3.38	1244.35	251.61	13.35	44.90
box-spheres	sah-exhaustive	3.84	342.40	51.83	13.35	11.73
dragon	sah-binning	6.69	52.39	7.97	10.68	10.53

Table 4.1: Per-pixel mean metrics (1024^2, 16 SPP).

SAH + binning yields the lowest triangle test counts. On bunny, SAH reduces tests from 413.77 (median) to 77.26 (**5.4**×). Exhaustive SAH takes 508 s vs. 4 s for binned, confirming the 8-bin approximation's efficiency. On dragon (871k triangles), exhaustive SAH would take 90 min while binned SAH completes in 6.69 s, avoiding the impracticality of naive rendering.

4.1 BVH Construction Metrics

Table 4.2 details the BVH trees after SAH construction. Dragon requires 6.69 s and produces 932,985 nodes for 871,322 triangles (depth 32, 1.9 prims/leaf). Build time is high due to single-threaded Python construction.

Scene	Nodes	Internal	Leaves	Depth	Prims/leaf
bunny	76,907	38,453	38,454	8.3	1.8
box-spheres	2,183	1,091	1,092	5.7	2.0
dragon	932,985	466,492	466,493	32	1.9

Table 4.2: BVH construction statistics (SAH + binning).

4.2 Render Time Logs

Table 4.3 breaks down pipeline timings on an NVIDIA RTX 3090. The GPU tonemapping executes in a single fused kernel (`postprocess_full_gpu_kernel`), combining exposure, the ACEScg LUT, and gamma correction. Without BVH the bunny scene requires 36 s. With SAH BVH and denoising the render time is 0.11 s at 16 SPP (BVH build time of 7.34 s excluded). Shadow rays represent 37–42% of all rays.

Stage	Time (s)
BVH build (bunny)	7.34
Render (bunny, w/ BVH, 16 SPP)	0.09
Tonemap + postprocess (GPU)	0.01
Denoise (GPU, OIDN RT)	0.02
Total render	**0.11**

Table 4.3: Enhanced Bunny scene render timing breakdown (RTX 3090, 1024^2, 16 SPP, denoise ON). BVH build time listed separately.

5 Visual Outputs

The figures below demonstrate the renderer's capabilities across various scenes. The bunny model showcases soft shadows and diffuse bouncing, while the advanced box highlights specular reflections, caustics, and transmission. The dragon scene tests the BVH traversal efficiency on highly complex geometry. All renders utilize stochastic jittered sampling and are processed through the OIDN denoiser, yielding smooth results even at a low sample count of 16 SPP.

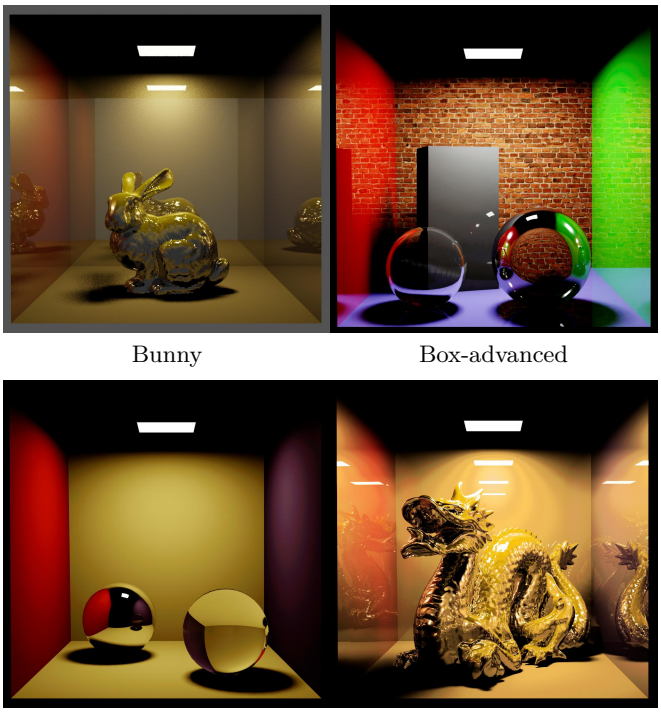

Figure 5.1: Scene renders (16 SPP, 1024^2, denoise ON).

5.1 Tonemapper Comparison

The renderer provides six tonemappers: *custom* (ACEScg → sRGB 3D LUT with Oklab hue compression [11], GPU), *Khronos PBR neutral* (per-channel glTF neutral), *Narkowicz* (per-channel ACES fit [8]), *Hill* (analytical ACES approximation [9]), *none* (hard clip), and *magenta* (debug overshoot).

Custom (Oklab 3D LUT [11]). The custom tonemapper bakes an entire ACEScg → sRGB pipeline into a 32^3 3D LUT spanning -10 to +10 EV. The offline pipeline applies: (i) CAT02 chromatic adaptation to convert ACEScg to linear sRGB. (ii) Asymmetrical crosstalk matrix simulating physical film emulsion where red bleeds into green, green into blue, and blue into red. (iii) Binary-search gamut compression in Oklab [11]: the algorithm bisects the chroma scale (a, b channels) for 15 iterations until no sRGB channel falls below zero. (iv) White highlight desaturation using a hermite smoothstep that gradually reduces chroma as lightness rises from 1.5 to 2.0, mimicking photographic highlight roll-off. (v) Uchimura piecewise tonemapping [12] applied to the achromatic norm with an exposure pre-scale of 0.6 (equivalent to the Academy RRT). The per-channel norm ratio is used to scale tonemapped values back to RGB, preserving hue during compression. (vi) sRGB gamma encode via 2.2 power. The resulting LUT is loaded at runtime and sampled with trilinear interpolation on the GPU kernel.

Khronos PBR neutral. Per-channel glTF neutral: clamps negatives, applies rational highlight compression. Lacks gamut compression and hue-aware processing.

Narkowicz / Hill (ACES [10]). Both apply the analytical ACES 1.0 fit per channel. Narkowicz gives punchy contrast with slight hue shifts. Hill offers smoother roll-off with reduced mid-tone contrast.

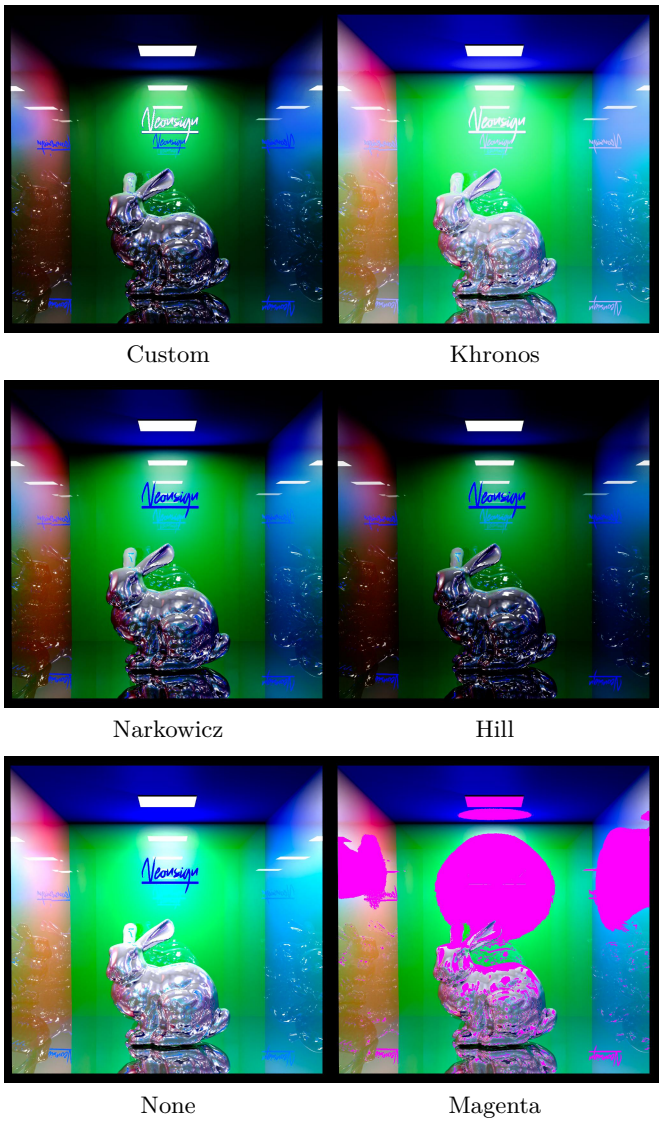

Figure 5.2: Tonemapper comparison (Enhanced bunny scene).

6 Conclusion

Ray workload analysis on the bunny scene (RTX 3090, BVH ON) shows primary rays at 30%, secondary at 32.8%, and shadow rays at 37.2%. The BVH traversal requires on average 14.7 node visits per triangle hit, with per-pixel intersection tests of 36–1379 (mean 115.9) vs. 279,967 without BVH. The *hard* ray fraction (>2× mean) is 8.6%, concentrated at specular highlights.

Sara demonstrates that a Python-based ray tracer with Numba CUDA achieves high-quality results with struct-of-arrays layout, SAH + binning BVH (5× fewer triangle tests vs. median splitting), and ACES colour pipeline. Future work includes GPU-side BVH construction (thrust/custom radix sort), volume rendering, and displacement mapping.

Failure cases. Several failure modes emerged: deep transmissive scenes cause ray path divergence that saturates the maximum bounce limit. BVH stacks exceeding `STACK_SIZE` = 64 silently truncate complex paths. The OIDN denoiser smears fine detail in low-SPP renders without normal/albedo buffers. Float32 precision causes self-shadowing artifacts despite epsilon corrections. On Ampere architecture the wavefront architecture's register pressure causes worse performance than the baseline megakernel due to register spilling to local memory.

References

[1] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3rd edition, 2016.

[2] Tomas Möller and Ben Trumbore. Fast, Minimum Storage Ray-Triangle Intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.

[3] NVIDIA Corporation. *CUDA C++ Programming Guide v12*. CUDA C++ Programming Guide, 2023.

[4] NVIDIA Corporation. *Nsight Compute—GPU Profiler for CUDA Applications*. NVIDIA Documentation, 2024.

[5] Intel Corporation. Intel Open Image Denoise: High-Performance Denoising Library for Ray Tracing. openimagedenoise.org, 2023.

[6] Antti Saavolainen. Ray Tracing Using GPU Computing, 2007. Master's thesis, Eastern Washington University. EWU Digital Commons

[7] Syoyo Fujita. TinyObjLoader: Tiny but Powerful Wavefront OBJ & MTL Loader. GitHub repository, 2023.

[8] Krzysztof Narkowicz. ACES Filmic Tone Mapping Curve. Knarkowicz Blog, 2016.

[9] Stephen Hill. ACES Filmic Tone Mapping Implementation in HLSL. BakingLab — ACES Tonemapping HLSL, 2016.

[10] The Academy of Motion Picture Arts and Sciences (AMPAS). SMPTE ST 2065-1: Academy Color Encoding Specification (ACES). ACES Central — SMPTE ST 2065-1, 2021.

[11] Björn Ottosson. Oklab: A perceptually uniform color space. Project page, 2020.

[12] Hajime Uchimura.
HDR theory and practice: Uchimura tone mapping curve. CEDEC 2017, Tokyo. Reference GLSL implementation.